FLOOD RELIEF MANAGEMENT SYSTEM

OOPs and DBMS – Final Project Report

Submitted by

NISHAL NARAYANAN T – 251001168

SANJAY R – 251001230

ROHITH BALAJI S – 251001218

SAIJAYNESH G – 251001225

of

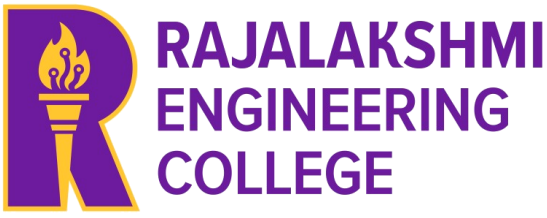
BACHELOR OF TECHNOLOGY

in

INFORMATION TECHNOLOGY

RAJALAKSHMI ENGINEERING COLLEGE, THANDALAM

(An Autonomous Institution)



NOV 2026

RAJALAKSHMI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"FLOOD RELIEF MANAGEMENT SYSTEM"** is the bonafide work of **"NISHAL NARAYANAN T (251001168), SANJAY R (251001230), ROHITH BALAJI S (251001218), SAIJAYNESH G (251001225)"** who carried out the project work under my supervision.

SIGNATURE

Dr. Priya Vijay

HEAD OF THE DEPARTMENT

Information Technology

Rajalakshmi Engineering College,

Rajalakshmi Nagar, Thandalam,

Chennai – 602 105

SIGNATURE

Janani M

COURSE INCHARGE

Information Technology

Rajalakshmi Engineering College,

Rajalakshmi Nagar, Thandalam,

Chennai – 602 105

This project report is submitted for the OOPs and DBMS Project held on ……..

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

We express our sincere gratitude to our Chairman, Chairperson and Principal of Rajalakshmi Engineering College for providing the facilities and a supportive environment to carry out this project.

We thank **Dr. P. Neelaveni**, Head of the Department of Information Technology, for her continuous encouragement and guidance.

We are deeply thankful to our Course Incharge, **Janani M**, Department of Information Technology, for her valuable guidance, timely suggestions and constant support throughout the course of this project.

We also thank all the faculty members of the department, our parents and our friends for their support and motivation during the development of the Flood Relief Management System.

NISHAL NARAYANAN T
SANJAY R
ROHITH BALAJI S
SAIJAYNESH G

ABSTRACT

Floods cause large-scale displacement of people and demand fast, well-coordinated relief. Many relief operations still depend on paper registers and spreadsheets, which leads to duplicate entries, lost records, uneven distribution of food and medicine, and poor visibility of how many people are sheltered in each camp. The **Flood Relief Management System** is a desktop application that automates the recording and tracking of flood victims, relief camps, volunteers, relief stock and distribution of supplies.

The system is developed in **Java** using core object-oriented concepts such as classes and objects, encapsulation, inheritance, abstraction, polymorphism, interfaces and exception handling. A **MySQL** relational database stores all data in normalized tables linked by primary and foreign keys, and the application connects to it through **JDBC**. The software supports complete **CRUD** operations (Create, Read, Update, Delete) with a Swing based graphical interface, secure admin login, search by ID, and printable reports. By centralising relief data, the system reduces manual paperwork, avoids duplicate aid, and helps administrators take quick and accurate decisions during a flood emergency.

Keywords: Flood Relief, Java, OOP, DBMS, JDBC, MySQL, CRUD, Swing.

TABLE OF CONTENTS

CHAPTER / CONTENTS **PAGE NO.**

CHAPTER 1: INTRODUCTION 7

1.1 PROBLEM STATEMENT 7

1.2 PROJECT OVERVIEW 7

1.3 PURPOSE 7

1.4 SCOPE 7

1.5 OBJECTIVES 7

CHAPTER 2: REQUIREMENTS AND FEASIBILITY 8

2.1 FUNCTIONAL REQUIREMENTS 8

2.2 NON-FUNCTIONAL REQUIREMENTS 8

2.3 HARDWARE AND SOFTWARE REQUIREMENTS 8

2.4 FEASIBILITY STUDY 9

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN 10

3.1 DATA FLOW DIAGRAM 10

3.2 USE CASE DIAGRAM 11

3.3 CLASS DIAGRAM 12

3.4 MODULE DESCRIPTION 12

CHAPTER 4: DATABASE DESIGN 13

4.1 ENTITY RELATIONSHIP DIAGRAM 13

4.2 TABLE STRUCTURES 14

4.3 NORMALIZATION 15

4.4 DBMS CONCEPTS USED 15

CHAPTER 5: IMPLEMENTATION 16

5.1 OOP CONCEPTS DEMONSTRATED 16

5.2 DATABASE CONNECTIVITY (JDBC) 16

5.3 MODEL CLASSES (INHERITANCE, ABSTRACTION, POLYMORPHISM) 17

5.4 CRUD OPERATIONS (DAO CLASS) 17

5.5 LOGIN MODULE 18

5.6 DISTRIBUTION WITH TRANSACTION 19

5.7 USER INTERFACE SCREENS 19

CHAPTER 6: VERIFICATION AND TEST PLAN **20**

6.1 TEST CASES 20

6.2 TESTING LEVELS 20

CHAPTER 7: SECURITY, LIMITATIONS, AND MAINTENANCE **21**

7.1 SECURITY 21

7.2 LIMITATIONS 21

7.3 MAINTENANCE 21

CHAPTER 8: CONCLUSION AND FUTURE WORK **22**

8.1 CONCLUSION 22

8.2 FUTURE WORK 22

APPENDIX A: SQL SCHEMA **24**

APPENDIX B: OUTPUT SCREENSHOTS **26**

CHAPTER 1: INTRODUCTION

1.1 PROBLEM STATEMENT

During floods, relief agencies must register affected families, allot them to camps, track volunteers and stock, and distribute supplies fairly. Manual record keeping is slow and error prone. There is a need for a computerised system that stores relief information centrally and makes it quickly accessible.

1.2 PROJECT OVERVIEW

The Flood Relief Management System is a Java desktop application backed by a MySQL database. It lets an authorised administrator log in, register flood victims, manage relief camps and volunteers, maintain stock of relief materials, record the distribution of items to victims, and view, search, update or delete any record.

1.3 PURPOSE

- To centralise flood victim, camp, volunteer and stock data in one database.
- To avoid duplicate registration and duplicate distribution of aid.
- To track camp occupancy against capacity and remaining stock of relief items.
- To demonstrate core Java OOP concepts, DBMS concepts, JDBC connectivity and CRUD operations.

1.4 SCOPE

- Secure admin login with validation.
- Registration and maintenance of victims, camps, volunteers and relief resources.
- Recording of distributions with automatic stock reduction.
- Search by ID, table view and print of records.
- Out of scope: GPS tracking, SMS alerts and a web or mobile front end (listed as future work).

1.5 OBJECTIVES

- Design a normalized relational schema with proper keys and constraints.
- Implement the application using object oriented design (classes, inheritance, interfaces, polymorphism).
- Connect Java to MySQL using JDBC and perform all CRUD operations safely using prepared statements.
- Test the system and document the results.

CHAPTER 2: REQUIREMENTS AND FEASIBILITY

2.1 FUNCTIONAL REQUIREMENTS

ID	Requirement	Description
FR1	Admin Login	The admin must authenticate with a username and password before using the system.
FR2	Register Victim	Add a victim with personal details, area affected and the assigned camp.
FR3	Manage Camps	Add, view, update and delete relief camps with capacity.
FR4	Manage Volunteers	Register volunteers and assign them to camps.
FR5	Manage Resources	Maintain the stock of food, water, medicine and clothing.
FR6	Distribute Relief	Record items given to a victim and reduce the stock.
FR7	View / Search	View all records in a table and search by ID.
FR8	Update / Delete	Modify or remove an existing record.

2.2 NON-FUNCTIONAL REQUIREMENTS

- **Usability:** simple, form based graphical interface.
- **Reliability:** data is stored persistently in MySQL.
- **Performance:** a record is added or retrieved in under a second for normal data sizes.
- **Security:** password protected login and parameterised SQL queries.
- **Maintainability:** modular classes with a DAO layer.

2.3 HARDWARE AND SOFTWARE REQUIREMENTS

Component	Specification
Processor	Intel Core i3 or higher
RAM	4 GB minimum
Storage	500 MB free space
Operating System	Windows 10 / 11, Linux or macOS
Language	Java (JDK 17 or later)
Database	MySQL 8.x
Connector	MySQL Connector/J (JDBC driver)
IDE	IntelliJ IDEA / Eclipse / NetBeans
GUI	Java Swing and AWT

2.4 FEASIBILITY STUDY

Technical feasibility: Java, Swing, JDBC and MySQL are mature, free and well documented, and the team has the skills to use them.

Economic feasibility: all tools are open source or free for education, so there is no licensing cost.

Operational feasibility: the interface is simple enough for relief staff with basic computer knowledge to use after short training.

Schedule feasibility: the scope is limited to core relief operations and can be completed within the academic term.

CHAPTER 3: SYSTEM ANALYSIS AND DESIGN

This chapter presents the flow of information and the design of the system using a data flow diagram, use case diagram and class diagram.

3.1 DATA FLOW DIAGRAM

A Data Flow Diagram (DFD) shows how data moves through a system. The diagram below represents the Flood Relief Management System.

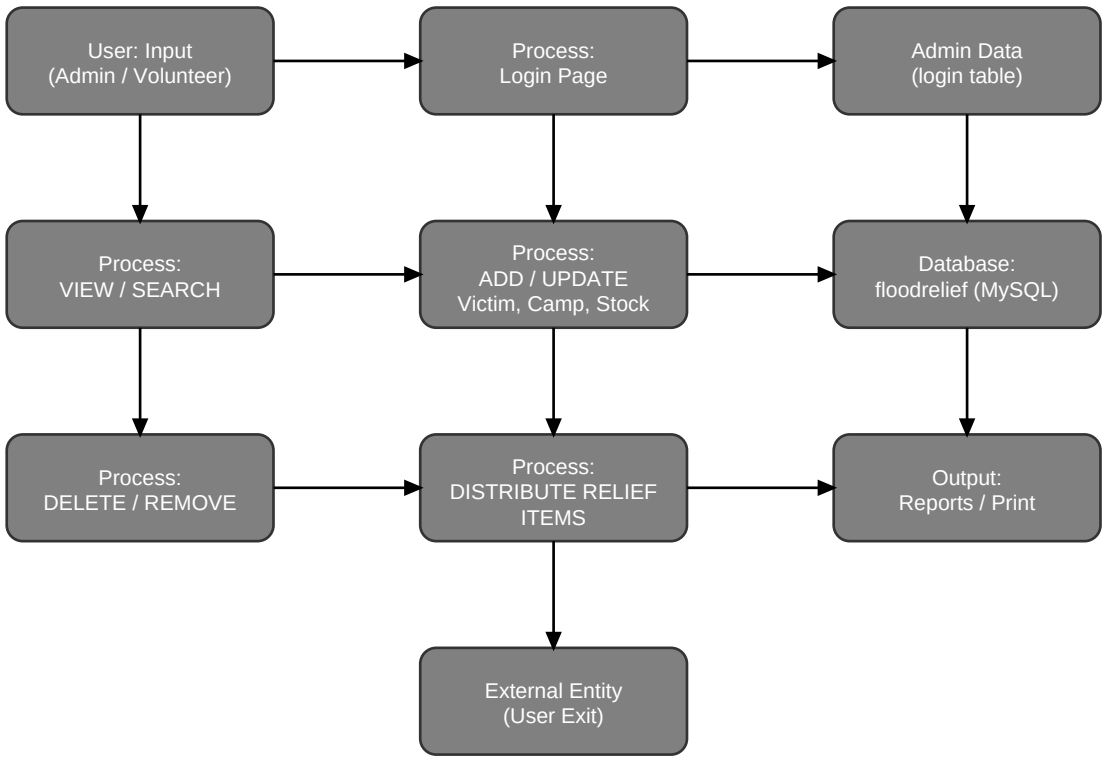


Figure 3.1 Data Flow Diagram

3.2 USE CASE DIAGRAM

A use case diagram depicts the interaction between actors and the system. The Admin has full access, while a Volunteer can log in, register victims, view records and record distributions.

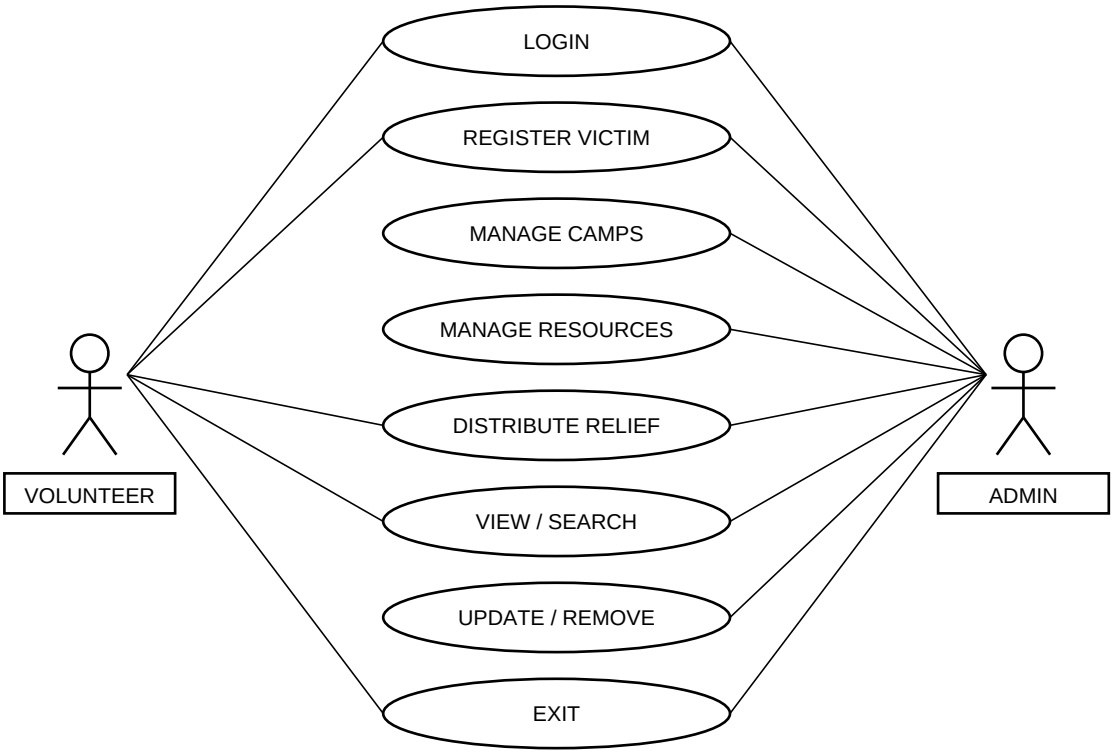


Figure 3.2 Use Case Diagram

Actor	Use cases
Admin	Login, Register Victim, Manage Camps, Manage Resources, Distribute Relief, View/Search, Update/Remove, Exit
Volunteer	Login, Register Victim, Distribute Relief, View/Search, Exit

3.3 CLASS DIAGRAM

The class diagram shows the main classes. *Person* is an abstract base class extended by *Victim* and *Volunteer*. *CrudOps* is an interface implemented by the DAO classes, and *DBConnection* is a singleton that provides the JDBC connection.

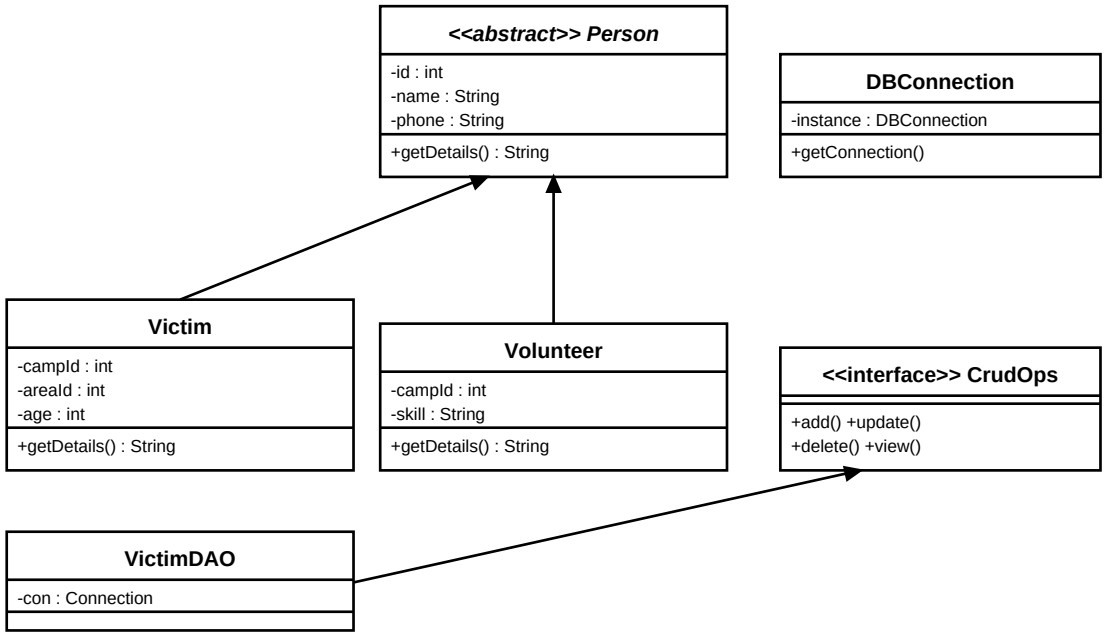


Figure 3.3 Class Diagram

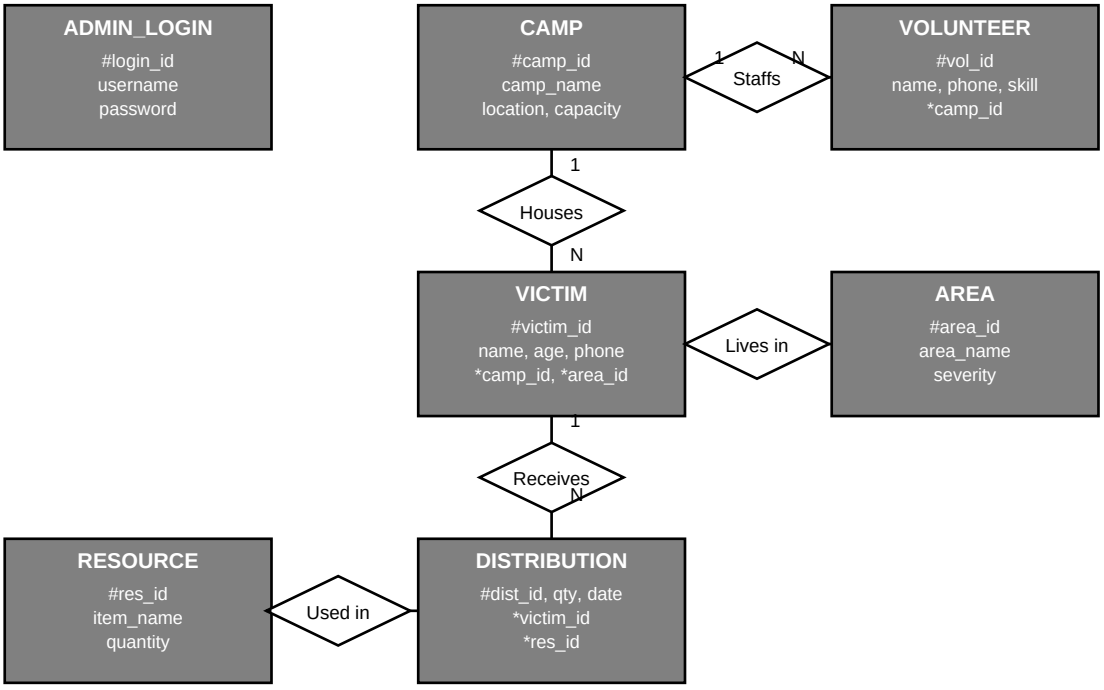
3.4 MODULE DESCRIPTION

Module	Description
Splash / Login	Shows the opening screen and validates the admin against the login table.
Victim	Adds a victim after generating a unique ID, assigns a camp and area, and supports view, update and delete.
Camp	Maintains camp name, location and capacity, and shows current occupancy.
Volunteer	Stores volunteer details and the camp each volunteer serves.
Resource	Keeps item names and available quantity.
Distribution	Records which item and quantity was given to which victim and reduces the stock.

CHAPTER 4: DATABASE DESIGN

4.1 ENTITY RELATIONSHIP DIAGRAM

The ER diagram shows the entities, attributes and relationships. A camp houses many victims and is staffed by many volunteers; a victim lives in one affected area and can receive many distributions; each distribution uses one resource item.



ER Diagram - Flood Relief Management System

*Figure 4.1 ER Diagram (# = primary key, * = foreign key)*

4.2 TABLE STRUCTURES

Table: login

Column	Data type	Constraint
login_id	INT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(30)	NOT NULL, UNIQUE
password	VARCHAR(64)	NOT NULL

Table: area

Column	Data type	Constraint
area_id	INT	PRIMARY KEY, AUTO_INCREMENT
area_name	VARCHAR(60)	NOT NULL
severity	VARCHAR(15)	CHECK (Low / Medium / High)

Table: camp

Column	Data type	Constraint
camp_id	INT	PRIMARY KEY, AUTO_INCREMENT
camp_name	VARCHAR(60)	NOT NULL
location	VARCHAR(100)	NOT NULL
capacity	INT	NOT NULL, CHECK (capacity > 0)

Table: victim

Column	Data type	Constraint
victim_id	INT	PRIMARY KEY
name	VARCHAR(50)	NOT NULL
age	INT	CHECK (age > 0)
phone	VARCHAR(15)	
family_members	INT	DEFAULT 1
camp_id	INT	FOREIGN KEY → camp(camp_id)
area_id	INT	FOREIGN KEY → area(area_id)

Table: volunteer

Column	Data type	Constraint
vol_id	INT	PRIMARY KEY, AUTO_INCREMENT
name	VARCHAR(50)	NOT NULL
phone	VARCHAR(15)	
skill	VARCHAR(40)	
camp_id	INT	FOREIGN KEY → camp(camp_id)

Table: resource

Column	Data type	Constraint
res_id	INT	PRIMARY KEY, AUTO_INCREMENT
item_name	VARCHAR(50)	NOT NULL, UNIQUE
quantity	INT	CHECK (quantity >= 0)

Table: distribution

Column	Data type	Constraint
dist_id	INT	PRIMARY KEY, AUTO_INCREMENT
victim_id	INT	FOREIGN KEY → victim(victim_id)
res_id	INT	FOREIGN KEY → resource(res_id)
qty	INT	CHECK (qty > 0)
dist_date	DATE	NOT NULL

4.3 NORMALIZATION

1NF: every column holds atomic values and each table has a primary key. **2NF:** all non-key attributes depend on the whole primary key. **3NF:** there are no transitive dependencies; camp, area and resource details are kept in their own tables and referenced by foreign keys, which removes redundancy such as repeating a camp location for every victim.

4.4 DBMS CONCEPTS USED

- Primary keys, foreign keys and referential integrity.
- NOT NULL, UNIQUE, CHECK and DEFAULT constraints.
- Joins to show a victim with the camp name and area name.
- Aggregate functions (COUNT, SUM) for camp occupancy and stock reports.
- Transactions: a distribution insert and the stock update are committed together.

Sample query (victims with their camp):

```
SELECT v.victim_id, v.name, c.camp_name, a.area_name
FROM victim v
JOIN camp c ON v.camp_id = c.camp_id
JOIN area a ON v.area_id = a.area_id;
```

CHAPTER 5: IMPLEMENTATION

The application is built with Java Swing for the interface, JDBC for database access and MySQL as the back end. The code is organised into model classes, DAO classes and GUI classes.

5.1 OOP CONCEPTS DEMONSTRATED

Concept	Where used
Class and Object	Victim, Camp, Volunteer, Resource objects represent database rows.
Encapsulation	Private fields with public getters and setters in every model class.
Inheritance	Victim and Volunteer extend the Person class.
Abstraction	Person is abstract and declares the abstract method getDetails().
Polymorphism	getDetails() is overridden in Victim and Volunteer (method overriding); runtime dispatch through a Person reference.
Interface	CrudOps declares add, update, delete and view, implemented by the DAO classes.
Exception handling	try-catch around all JDBC calls, with user friendly dialog messages.
Singleton pattern	DBConnection returns a single shared connection.
Collections	ArrayList is used to hold lists of records loaded from the database.

5.2 DATABASE CONNECTIVITY (JDBC)

The *DBConnection* class loads the MySQL driver and opens the connection once. All DAO classes obtain the connection from it.

```
package floodrelief;
import java.sql.*;

public class DBConnection {
    private static DBConnection instance;
    private Connection con;

    private DBConnection() {
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            con = DriverManager.getConnection(
                "jdbc:mysql://localhost:3306/floodrelief", "root", "your_password");
        } catch (Exception e) { e.printStackTrace(); }
    }
    public static DBConnection getInstance() {
        if (instance == null) instance = new DBConnection();
        return instance;
    }
    public Connection getConnection() { return con; }
}
```

5.3 MODEL CLASSES (INHERITANCE, ABSTRACTION, POLYMORPHISM)

```
package floodrelief;

public abstract class Person {
    private int id;
    private String name;
    private String phone;

    public Person(int id, String name, String phone) {
        this.id = id; this.name = name; this.phone = phone;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public String getPhone() { return phone; }
    public abstract String getDetails();          // abstraction
}

public class Victim extends Person {              // inheritance
    private int age, campId, areaId;
    public Victim(int id, String name, String phone, int age, int campId, int areaId)
        {
        super(id, name, phone);
        this.age = age; this.campId = campId; this.areaId = areaId;
    }
    public int getAge() { return age; }
    public int getCampId() { return campId; }
    public int getAreaId() { return areaId; }
    @Override
    public String getDetails() {                  // polymorphism
        return "Victim " + getId() + ": " + getName() + ", Camp " + campId;
    }
}

public class Volunteer extends Person {
    private String skill; private int campId;
    public Volunteer(int id, String name, String phone, String skill, int campId) {
        super(id, name, phone); this.skill = skill; this.campId = campId;
    }
    @Override
    public String getDetails() {
        return "Volunteer " + getId() + ": " + getName() + " (" + skill + ")";
    }
}

public interface CrudOps<T> {                      // interface
    void add(T t) throws Exception;
    void update(T t) throws Exception;
    void delete(int id) throws Exception;
    java.util.List<T> view() throws Exception;
}
```

5.4 CRUD OPERATIONS (DAO CLASS)

The *VictimDAO* class implements *CrudOps* and performs Create, Read, Update and Delete using **PreparedStatement**, which prevents SQL injection.

```
package floodrelief;
import java.sql.*;
import java.util.*;
public class VictimDAO implements CrudOps<Victim> {
```

```
    private Connection con = DBConnection.getInstance().getConnection();

    @Override                                                    // CREATE
    public void add(Victim v) throws Exception {
        String sql = "INSERT INTO victim(victim_id,name,age,phone,camp_id,area_id)
            VALUES(?,?,?,?,?,?)";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setInt(1, v.getId());   ps.setString(2, v.getName());
            ps.setInt(3, v.getAge());  ps.setString(4, v.getPhone());
            ps.setInt(5, v.getCampId()); ps.setInt(6, v.getAreaId());
            ps.executeUpdate();
        }
    }
    @Override                                                    // READ
    public List<Victim> view() throws Exception {
        List<Victim> list = new ArrayList<>();
        ResultSet rs = con.createStatement().executeQuery("SELECT * FROM victim");
        while (rs.next())
            list.add(new Victim(rs.getInt("victim_id"), rs.getString("name"),
                rs.getString("phone"),
                                rs.getInt("age"), rs.getInt("camp_id"),
                                    rs.getInt("area_id")));
        return list;
    }
    @Override                                                    // UPDATE
    public void update(Victim v) throws Exception {
        String sql = "UPDATE victim SET phone=?, camp_id=?, area_id=? WHERE
            victim_id=?";
        try (PreparedStatement ps = con.prepareStatement(sql)) {
            ps.setString(1, v.getPhone()); ps.setInt(2, v.getCampId());
            ps.setInt(3, v.getAreaId());   ps.setInt(4, v.getId());
            ps.executeUpdate();
        }
    }
    @Override                                                    // DELETE
    public void delete(int id) throws Exception {
        try (PreparedStatement ps = con.prepareStatement("DELETE FROM victim WHERE
            victim_id=?")) {
            ps.setInt(1, id); ps.executeUpdate();
        }
    }
}
```

5.5 LOGIN MODULE

```
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == login) {
        try {
            String sql = "SELECT * FROM login WHERE username=? AND password=?";
            PreparedStatement ps =
                DBConnection.getInstance().getConnection().prepareStatement(sql);
            ps.setString(1, tusername.getText());
            ps.setString(2, new String(tpassword.getPassword()));
            ResultSet rs = ps.executeQuery();
            if (rs.next()) { setVisible(false); new MainClass(); }
            else JOptionPane.showMessageDialog(null, "Invalid username or password");
        } catch (Exception ex) { ex.printStackTrace(); }
    } else System.exit(0);
}
```

5.6 DISTRIBUTION WITH TRANSACTION

```
Connection con = DBConnection.getInstance().getConnection();
try {
    con.setAutoCommit(false);
    PreparedStatement a = con.prepareStatement(
        "INSERT INTO distribution(victim_id,res_id,qty,dist_date)
            VALUES(?,?,?,CURDATE())");
    a.setInt(1, victimId); a.setInt(2, resId); a.setInt(3, qty); a.executeUpdate();
    PreparedStatement b = con.prepareStatement(
        "UPDATE resource SET quantity = quantity - ? WHERE res_id=? AND quantity >=
            ?");
    b.setInt(1, qty); b.setInt(2, resId); b.setInt(3, qty);
    if (b.executeUpdate() == 0) throw new Exception("Insufficient stock");
    con.commit();
} catch (Exception ex) { con.rollback(); JOptionPane.showMessageDialog(null,
    ex.getMessage()); }
finally { con.setAutoCommit(true); }
```

5.7 USER INTERFACE SCREENS

- **Splash page:** opening screen shown for a few seconds.
- **Login page:** username and password with LOGIN and BACK buttons.
- **Home page:** buttons for Add Victim, View, Update, Remove, Camps, Volunteers, Resources and Distribution.
- **Add page:** form with a system generated victim ID.
- **View page:** table of records with search by ID, Print, Update and Back.
- **Update and Remove pages:** load a selected record, then update or delete it after confirmation.

Screenshots of each screen are given in Appendix B.

CHAPTER 6: VERIFICATION AND TEST PLAN

Testing checks that every module meets its requirement. Black box test cases were designed for each CRUD operation and for the login.

6.1 TEST CASES

ID	Module	Input / Action	Expected result	Status
TC1	Login	Valid username and password	Home page opens	Pass
TC2	Login	Wrong password	Message: Invalid username or password	Pass
TC3	Add Victim	All fields filled, click ADD	Record saved; success message	Pass
TC4	Add Victim	Duplicate victim ID	Insert rejected by primary key; error handled	Pass
TC5	Add Victim	Invalid camp ID	Rejected by foreign key constraint	Pass
TC6	View	Open View page	All records shown in table	Pass
TC7	Search	Select ID and click Search	Only that record shown	Pass
TC8	Update	Change phone and camp, click Update	Record changed in database	Pass
TC9	Delete	Select ID, click Delete	Record removed; confirmation shown	Pass
TC10	Distribution	Quantity greater than stock	Transaction rolled back; “Insufficient stock”	Pass
TC11	Distribution	Valid quantity	Distribution saved and stock reduced	Pass
TC12	Camp	Capacity set to 0	Rejected by CHECK constraint	Pass

6.2 TESTING LEVELS

- **Unit testing:** each DAO method tested separately against the database.
- **Integration testing:** GUI, DAO and database tested together.
- **System testing:** complete flow from login to distribution and exit.
- **Validation:** empty fields and invalid numbers are checked before saving.

Note: the Status column should be updated with the results of your own test run before submission.

CHAPTER 7: SECURITY, LIMITATIONS, AND MAINTENANCE

7.1 SECURITY

- Only users with valid credentials can enter the system.
- PreparedStatement is used for queries to prevent SQL injection.
- The password field masks input (JPasswordField).
- Foreign keys and CHECK constraints protect data integrity.
- Passwords should be stored as hashes (for example SHA-256) rather than plain text.
- The database account should have only the privileges the application needs.

7.2 LIMITATIONS

- Desktop application; it runs on one machine and has no web or mobile access.
- No real time alerts or SMS notifications to victims.
- Single admin role; fine grained role based access is not implemented.
- No map or GPS integration to show flood affected zones.
- Database credentials are configured in the source code.

7.3 MAINTENANCE

- **Corrective:** fixing defects found during use.
- **Adaptive:** updating for new Java or MySQL versions.
- **Perfective:** adding new reports and features.
- **Preventive:** regular database backups and code cleanup.
- Modular classes and the DAO layer make changes easy without affecting other parts of the system.

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 CONCLUSION

The Flood Relief Management System successfully automates the management of flood victims, relief camps, volunteers, stock and distribution. Core Java OOP concepts – classes, encapsulation, inheritance, abstraction, polymorphism, interfaces and exception handling – were applied in the design. DBMS concepts such as normalization, keys, constraints, joins and transactions were used in the MySQL database, and JDBC provided smooth connectivity between the application and the database. All CRUD operations work through an easy to use interface.

Through this project the team gained practical experience in object oriented design, database design, JDBC programming, testing and documentation, and learned how software can support disaster relief by reducing delay and error.

8.2 FUTURE WORK

- Convert the system into a web or mobile application so field workers can update data on site.
- Add SMS and email alerts for relief updates and flood warnings.
- Integrate maps and GPS to locate affected areas and nearest camps.
- Add role based access and encrypted password storage.
- Generate dashboards and PDF reports for authorities.
- Add donor management and online donation tracking.

REFERENCES

[1] Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill / Oracle Press.

[2] Oracle, *The Java Tutorials – JDBC Database Access*, https://docs.oracle.com/javase/tutorial/jdbc/

[3] Oracle, *Java Swing Tutorial*, https://docs.oracle.com/javase/tutorial/uiswing/

[4] MySQL Documentation, https://dev.mysql.com/doc/

[5] Abraham Silberschatz, Henry Korth and S. Sudarshan, *Database System Concepts*, McGraw-Hill.

[6] Ramez Elmasri and Shamkant Navathe, *Fundamentals of Database Systems*, Pearson.

APPENDICES

APPENDIX A: SQL SCHEMA

```
CREATE DATABASE floodrelief;
USE floodrelief;

CREATE TABLE login (
  login_id INT PRIMARY KEY AUTO_INCREMENT,
  username VARCHAR(30) NOT NULL UNIQUE,
  password VARCHAR(64) NOT NULL
);

CREATE TABLE area (
  area_id   INT PRIMARY KEY AUTO_INCREMENT,
  area_name VARCHAR(60) NOT NULL,
  severity  VARCHAR(15) CHECK (severity IN ('Low','Medium','High'))
);

CREATE TABLE camp (
  camp_id   INT PRIMARY KEY AUTO_INCREMENT,
  camp_name VARCHAR(60)  NOT NULL,
  location  VARCHAR(100) NOT NULL,
  capacity  INT NOT NULL CHECK (capacity > 0)
);

CREATE TABLE victim (
  victim_id      INT PRIMARY KEY,
  name           VARCHAR(50) NOT NULL,
  age            INT CHECK (age > 0),
  phone          VARCHAR(15),
  family_members INT DEFAULT 1,
  camp_id        INT,
  area_id        INT,
  FOREIGN KEY (camp_id) REFERENCES camp(camp_id),
  FOREIGN KEY (area_id) REFERENCES area(area_id)
);

CREATE TABLE volunteer (
  vol_id  INT PRIMARY KEY AUTO_INCREMENT,
  name    VARCHAR(50) NOT NULL,
  phone   VARCHAR(15),
  skill   VARCHAR(40),
  camp_id INT,
  FOREIGN KEY (camp_id) REFERENCES camp(camp_id)
);

CREATE TABLE resource (
  res_id    INT PRIMARY KEY AUTO_INCREMENT,
  item_name VARCHAR(50) NOT NULL UNIQUE,
  quantity  INT CHECK (quantity >= 0)
);

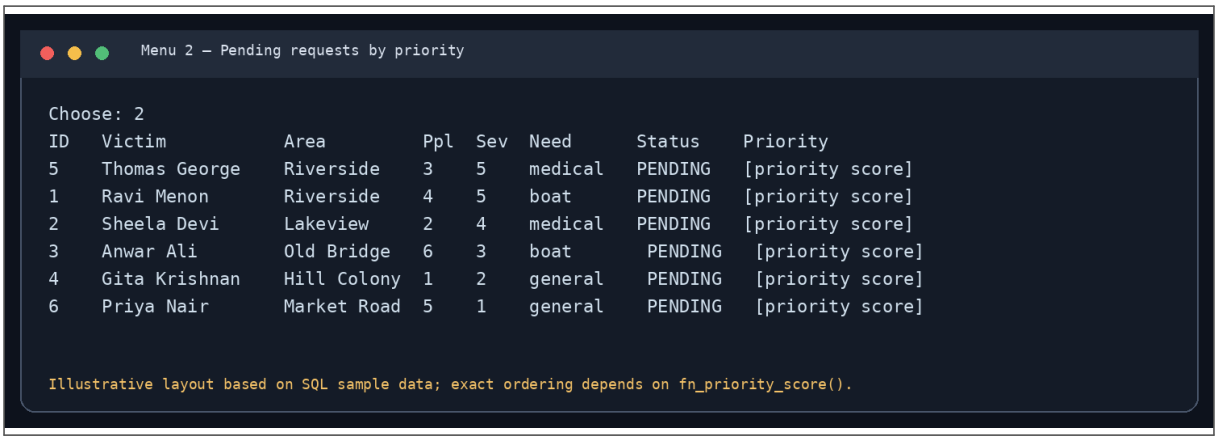
CREATE TABLE distribution (
  dist_id   INT PRIMARY KEY AUTO_INCREMENT,
  victim_id INT,
  res_id    INT,
  qty       INT CHECK (qty > 0),
  dist_date DATE NOT NULL,
  FOREIGN KEY (victim_id) REFERENCES victim(victim_id),
  FOREIGN KEY (res_id)    REFERENCES resource(res_id)
);
```

```
INSERT INTO login(username,password) VALUES ('admin','admin123');
```

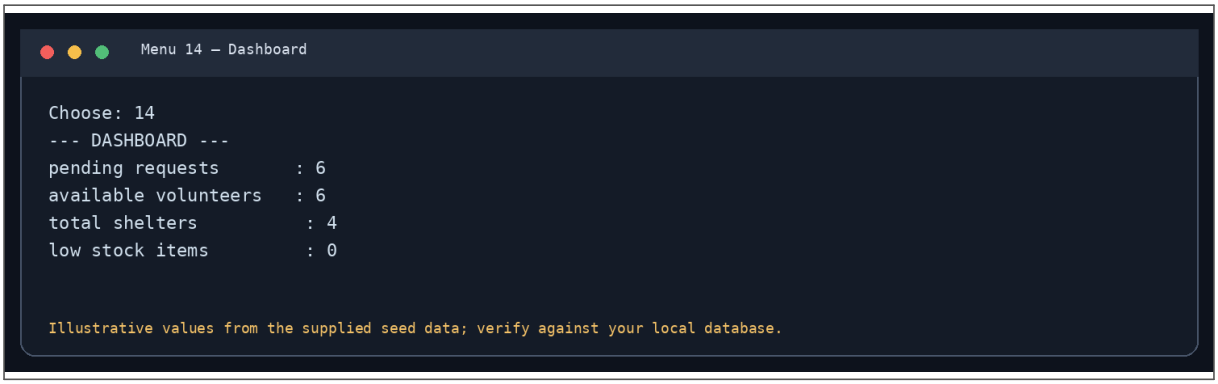
APPENDIX B: OUTPUT SCREENSHOTS



B.1 Main Menu – Disaster / Flood Relief Coordinator



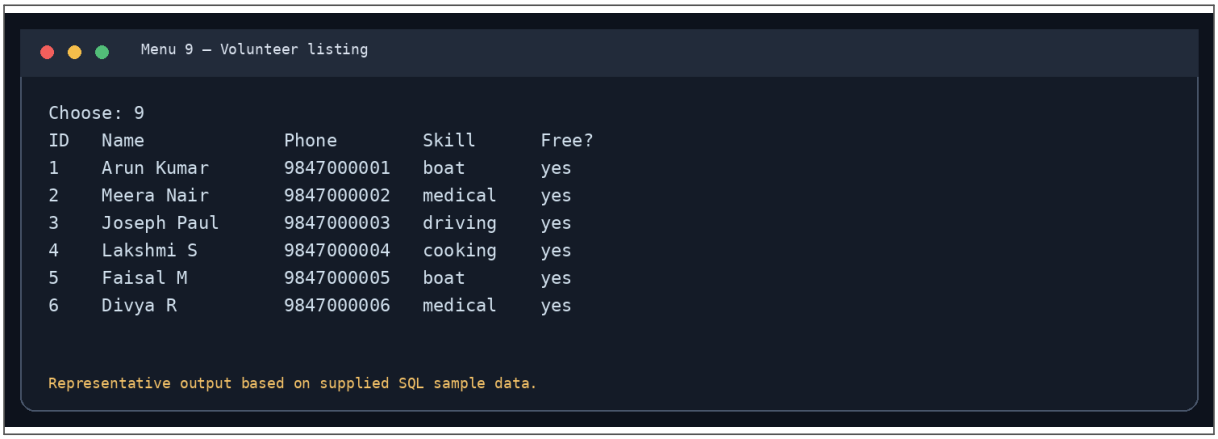
B.2 Pending Rescue Requests by Priority (Menu 2)



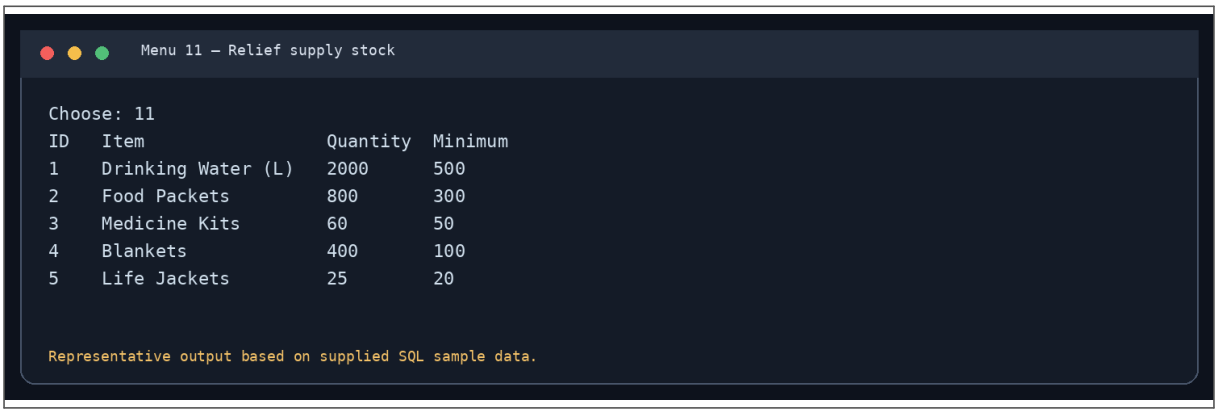
B.3 Dashboard Report (Menu 14)



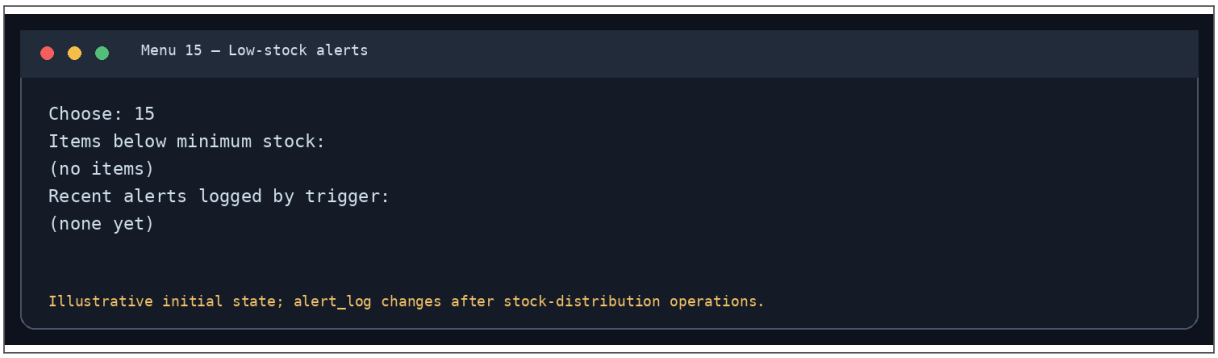
B.4 Shelter Listing (Menu 7)



B.5 Volunteer Listing (Menu 9)



B.6 Relief Supply Stock (Menu 11)



B.7 Low-Stock Alerts (Menu 15)